Project demo

This is a demonstration of the main system behaviours, it connects to the methods and result of the Project Report. Please run these commands in the repos root.

Introduction

The project builds an inverted index over Reuters 21578 corpus. The index maps a term to the DocID that contains it. I implemented a naive indexer that sorts and groups, and an in memory SPIMI indexer, then compared the twos accuracy and construction times. We also experimented how preprocessing changes the indexes statistics and query results.

Lets go through our main modules:

- Reader.py: Reads the corpuses 22 .sgm files and ensures preservation of each articles NEWID
- tokenizer.py: Tokenizes the text and applies normalization
- pairs.py , naive_indexer.py and spimi_indexer.py implement the two indexing.
- query.py ; uses the single term lookup and the AND intersection.
- compress.py : runs the whole preprocessing experiment (Check Compression experiment and Project Report)
- bench.py: it compares both indexing times over the same 10,000 pairs
- demo.py provides short commands

To see changes more concretely I made two small articles to read, so it's easy to inspect the pairs and postings. This test shows that the two indexes produce the same index and the AND query only gives back IDs that occur in both terms posting

To Run, run this from the repos root:

- python demo.py tiny

Reading and tokenization

Reader.py and tokenizer.py :
The reader goes through the .sgm files in sorted order, decodes them as latin-1 and then extracts the NEWID, title, and finally body. The tokenizer then extracts the ASCII letters and digits within the text. Both indexing and qwerty processing use this same tokenizer so terms are treated the same consistently.

Comparing the indexers

The naive indexer materializes the term document pairs, then removes any duplicates and sorts them, which then groups them into a posting. While SPIMI adds IDs directly to each term postings list during the consumption of pairs in ascending DOCid order, this process allows the indexer to avoid global sort.

We generate the first 10K pairs before timing, alternating execution order over 7 runs, checking that the indexes match and then calculate and report median build times, leaving tokenization and parsing out of the timings.

The file bench.py calls the spimi_indexer.py and run_naive.py building function, if you'd only like to compare the two it requires only to run the bench.py file but if you'd like to see the full corpus build run run_naive.py as well.

Run:

- python run_naive.py
- python bench.py

Queries

A single term query should return those terms postings. And an AND query intersects sorted postings with a two point algo, so only documents having both terms remain.

Run:

- Python demo.py queries

Preprocessing

"Compression experiment" file talks about it in detail
Run:

- python demo.py compression

Tests

The tests are there to check the tiny corpus index, showcasing the equality between the 2 indexers, query behavior, preprocession and posting list intersections.

Run:

- python -m unittest test_tiny.py